

2026-09-30

# Native GeoArrow Computation Engine in R

Josiah Parry 

*This template was last revised on 2026-03-24.*

## Executive Summary

This proposal seeks to ameliorate three challenges facing the R-spatial ecosystem.

First, spatial analysis in R “is slow” and can be dramatically improved. The de facto package for vector-based spatial analysis in R is the `{sf}` package. Due to its ergonomic API and its integration with the tidyverse, it is the common choice for spatial data analysis. However, `{sf}` can struggle to scale effectively and has caused developers to turn to other tools and other languages.

Secondly, the spatial ecosystem has been quick to adopt GeoArrow, an extension of the Apache Arrow standard that defines how spatial data can be represented in memory. The standardization on (Geo)Arrow enables the development of packages that can work on in-memory data without incurring any additional overhead costs of deserializing and serializing data into intermediate objects.

Lastly, concurrent to the development of GeoArrow has been the development of GeoParquet. At present, GeoParquet support in R has focused on consuming the format rather than producing it.

I seek \$5,000 USD to fund the completion of the development of a new Rust-based, GeoArrow native, and highly performant spatial computation engine for R called `{geoarrows}`. The R package will provide:

- GeoArrow-native and parallelized computation engine
- Persistent spatial RTree and KDTree spatial indexes for efficient spatial querying and joining
- Drop in compatibility with `{sf}`
- Enhanced GeoParquet support such as spatial sorting, bounding boxes on row-chunks for predicate pushdown, and persisting coordinate reference systems.

## Signatories

### Project team

Josiah Parry is a Sr. Product Engineer in Spatial Statistics at Esri.

## Contributors

## Consulted

## The Problem

### The (de)serialization problem

While `{sf}` is a foundational package for the R ecosystem, its performance has failed to scale effectively as the size of data that people commonly work with has grown. This is a common experience. Users report [slow distance calculations on modest data](#), [write their own batching workarounds to finish jobs at all](#), and [performance](#) remains an open topic in `{sf}`'s own issue tracker.

[duckspatial's benchmark](#) indicates that it is roughly 50 times faster than `{sf}` on a spatial join. The [geoarrowr's replication](#) of the same benchmark finds that it is roughly 6 times faster than `{duckspatial}` on that operation.

The biggest challenge with `{sf}` is primarily the fact that geometries are stored in R as a list. Each operation that `{sf}` performs has to be first converted to a GEOS or S2 geometry. The S2 or GEOS libraries perform operations on their native geometries and then `{sf}` captures their results and again converts them back to their R representation. This process of converting to and from is the expensive (de)serialization.

GeoArrow as a standard exists to address exactly this problem. It has been a focus of [Spatial Data Science Across Languages](#) since its inception in 2024 and [again in 2025](#), and as a result we are seeing coordinated adoption across all languages. As an indicator of its importance, GDAL added a [column-oriented Arrow API](#). GeoPandas, too, added [native GeoArrow support](#).

At present, R support is limited. The R package `{geoarrow}` provides a `{nanoarrow}` and `{arrow}` compatible vector representation. The ISC-funded `{geoarrowWidget}` package assists in transporting `{geoarrow}` vectors to the browser. However, the R community lacks a GeoArrow-native compute engine that can scale effectively.

### Alternative Computation Engines

`{geos}` has historically been the fastest R package in the ecosystem but is very low level and thus not very widely used.

Currently, there are two primary options for scalable geospatial computation engines in R: 1) `{duckspatial}` and 2) `{sedonadb}`. Neither works with native `data.frames` and both require external downloads.

The `duckspatial` R package uses the [spatial extension](#) to DuckDB, which must be downloaded and installed separately. On an M5 MacBook Pro that extension is 57mb. It also requires [static linking](#).

`{sedonadb}` provides bindings to [Apache Sedona](#) and is arguably the most scalable of the options. The installation is 261mb. It is also not available on CRAN, lacks sufficient documentation, and has a bespoke API, all of which limit adoption.

By comparison, `{geoarrowr}` in its current state is a 14mb installation with no external downloads and no system dependencies. Even `{sf}`, which relies on GDAL, GEOS, and PROJ, is 88mb installed.

### GeoParquet

[GeoParquet](#) is becoming the standard for storing and sharing spatial data. The `{geoarrow}` R package provides GeoParquet metadata to the `{arrow}` R package so that `{arrow}` can read GeoParquet natively. Writing can be done via `arrow::write_parquet()`, and `{sfarrow}` delegates to `{arrow}` directly. This is minimally viable GeoParquet.

Parquet's benefit is that row-group metadata can be used to efficiently filter data. At present there is no easy way to record spatial metadata in a GeoParquet file from R. Geometries are written in the order that they appear in the data.frame. Spatial proximity is entirely ignored and thus row-groups are not spatially organized, meaning libraries cannot spatially filter data without ingesting all of it. There are [current efforts](#) to formalize the benefits of row sorting for geospatial data and integrate them with GeoParquet. Lastly, no R library writes projjson, so the CRS of a file written from R is lost to downstream tools.

We need an R package with a modest footprint, no system dependencies, and no external downloads, that works on GeoArrow natively, provides fast, scalable, and efficient spatial operations, and offers true GeoParquet support.

## The proposal

### Overview

The `{geoarrows}` package is a GeoArrow-native spatial computation engine designed for ergonomic use within the R ecosystem. The package is in the middle of its development life cycle and this grant would fund its completion.

Today, the package provides:

- performant readers for shapfile, GeoJson, and FlatGeobuf
- R-tree and KD-tree spatial indexes
- Spatial, nearest neighbor, and distance band joins
- User defined functions registered with `{arrow}`

This package aims to be faster than `geos`, competitive with `duckspatial` and `sedonadb`, and as ergonomic as `{sf}` (with a drop in replacement via `mask_sf()` for **in memory** computation).

Completion of the package is split into four milestones:

1. Complete the core geometry engine so that every algorithm provided by the Rust crate `geo` is available from R.
2. Create spatial indexes that can be written to disk and used to spatially sort geometries.
3. Provide drop in `{sf}` compatibility via `mask_sf()` and release the package on CRAN.
4. Persist coordinate reference systems so that GeoArrow transfer between R, Python, DuckDB, and GDAL is lossless.

### Prior art

The `{geoarrows}` package is the culmination of years of exploring the intersection of Rust, geospatial, and GeoArrow. Initially, the `{rsgeo}` package was created to be a zero-dependency and high-performance spatial computation engine with [Rust based conversion](#) to and from `{sf}`. As GeoArrow began to pick up, the `{anime}` line matching algorithm became a proof of concept for its use in R, built on my [arrow-extendr](#) Rust library which provides FFI between R and Rust for `{arrow}`, `{nanoarrow}`, `{polars}`, and `{geoarrow}`. The [initial version](#) of this package was started in 2024, but paused due to an unstable `geoarrow-rust` API and challenges with Arrow FFI that are now resolved.

## Detail

### Minimum Viable Product

A minimal viable product provides the core geometry implementations on geoarrow arrays in R. The state of the R package today is nearly an MVP. With this funding it can be improved to be a high quality spatial package for the R ecosystem.

### Architecture

The `{geoarrowr}` package is a Rust-powered R package built on `extendr`. Arrow to Rust FFI is handled via the `arrow-extendr` crate that I built and maintain. The R representation of `GeoArrow` arrays leans on the implementation of the `{geoarrow}` R package. Native algorithms use the `geo` Rust crate via the constant-time access via the `geo-traits` crate. Parallelization of geometric operations is handled via the `rayon` crate. In GeoArrow, coordinate reference systems (CRS) must be represented as `projjson`. For this, a new Rust crate, `projjson`, was prototyped and published. Milestone 4 integrates it into `{geoarrowr}`. Spatial indexes are created and persisted using the `geo-index` Rust crate.

### Assumptions

There are no outstanding assumptions that can invalidate the concept proposed here. It has been validated over three years of experimentation and development in the geospatial and Rust environments.

### External dependencies

`{geoarrowr}` has no external system dependencies. It does not link GDAL, GEOS, PROJ, DuckDB, nor are there any runtime downloads. Dependencies are pure-Rust that are only required at build time. CRAN will build and provide package binaries.

- Imports: `cli`, `rlang`
- Suggests: `arrow`, `dplyr`, `geoarrow`, `nanoarrow`, `sf`, `wk`, `testthat`, and `withr`

The Rust crates are named in the architecture section above and enumerated in the package's `Cargo.toml`.

### Limitations

The core library for geometric operations works only on 2D geometries meaning any operations on geometries with Z or M values is silently lost.

The CRAN tarball size and the limited support for `projjson` in the R ecosystem are also limitations. Both are addressed in Milestones 3 and 4 respectively.

## Project plan

### Start-up phase

`{geoarrowr}` is already scaffolded and ready to continue development. The repository is configured under the MIT license, with continuous integration running. Work on the first milestone is ready to commence upon award.

## Technical delivery

Upon award, a milestone tracker will be included in the project's README similar to the existing [README.md](#) which tracks geo compatibility. This will be migrated to the root of the repo with a detailed list that is associated with each milestone.

## Other aspects

A formal CRAN release will indicate completion of the project followed by a join blog post with the R Consortium.

Upon successful completion, the `mask_sf()` capability will inform completion and development of the `{sdf}` R package which provides a geometry-agnostic interface. I will contact the R-spatial organization and ask if the package can be formally integrated into the ecosystem.

## Budget & funding plan

I request \$5,000 USD to fund the development of `{georowrs}`.

### Milestone 1 - Complete Core Geometry Engine Implementation (\$1,500)

This milestone funds the development effort to complete the core computation engine in `{georowrs}` such that there is near-complete parity with `{sf}`. To accomplish the milestone, the following will be completed:

- Geometry reparation: `st_make_valid()`
- Geometry type checking: `st_geometry_try()` and `st_is()`
- Collection extraction: `st_collection_extract()`
- Perimeter calculation: `st_perimeter()`
- Coordinate comparison: `st_equals_exact()`
- Jittering: `st_jitter()`
- Winding: `st_reverse()`
- Areal interpolation: `st_interpolate_aw()`
- Bounding box: `st_bbox()`
- Polygon constructors:

The following features are out of scope of this proposal as there are no pure rust alternatives to rely on at present.

- 3D geometries
- Snapping
- Polygonization
- Geometry Projection

Completion is expected within 2 months of the award date.

### Milestone 2 - Persistent Spatial Indexes and Spatial Sorting (\$1,000)

This milestone funds the development of creating and persisting spatial RTree and KDTree spatial indexes. These spatial indexes are created on the fly for each operation performed by `{sf}` as well as other libraries. For operations that require repeated look ups, persisting the index can be highly efficient.

Further, these indexes can be used to perform sorting of geometries to ensure that nearby geometries are ordered together. Spatial sorting greatly improves the effectiveness of row group statistics in GeoParquet files, which is what makes partial reads possible.

For spatial sorting, Hilbert and sort-tile-recursive (STR) will both be supported. `{geos}` provides an STR tree, but it cannot be persisted to disk or used to sort geometries, and no other R package, to my knowledge, provides either capability.

Completion is expected within 3 months of the award date.

### **Milestone 3 - `{sf}` Compatibility and CRAN release (\$1,500)**

In order for the package to be useful to the broadest community, this milestone will create drop in support for `{sf}` via S3 namespace masking. Users can call `mask_sf()` to overload the `{sf}` namespace with generics that enable `{sf}` to work with `{geoarrowrs}` geometries.

Following, documentation, examples, and benchmarks will be tidied up, and rewritten for ease of adoption.

One **risk** is that the source tarball is 14mb which is 9mb larger than the requested CRAN limit. However, exceptions can be made. For example, the prior art `{rsgeo}` was a [9mb tarball](#). And other Rust based R packages such as `{fcl}`, are [15mb](#) and supported on CRAN.

One **limitation** is that the `geo` crate that we rely on geometry operations does not have full spherical support whereas `{sf}` does via the `{s2}` package. In the event that a spherical CRS is used, geodesic algorithms will be used wherever possible. Otherwise, an error will be raised. `{geoarrowrs}` will not silently return a planar result for geographic coordinates—an issue [documented with painstaking detail](#).

Completion is expected within 5 months of the award date.

### **Milestone 4 - GeoArrow and GeoParquet metadata (\$1,000)**

At present, `geoarrowrs` cannot propagate a coordinate reference system (CRS) across the R to Rust FFI boundary. Milestone 3 is possible by leaning on `{sf}`'s CRS object and adding it as an attribute on the `geoarrow` vectors which requires PROJ to be installed. This milestone would ensure that no external system dependencies would be needed.

Further, the GeoArrow specification records CRS as column level metadata in the projjson format. To support this, the `projjson-rs` crate was created which provides strongly typed reading and writing of `projjson` with no external system dependencies. As a result, GeoArrow transferred from DuckDB, R, Python, and GDAL would propagate the CRS correctly.

The column level metadata is also utilized by GeoParquet. As such, this milestone completely implements the GeoParquet 1.1 column metadata: `encoding`, `geometry_types`, `crs`, `bbox`, `covering`, `orientation`, `edges`, and `epoch`.

Of these, **covering** matters most. It records a bounding box column whose Parquet statistics allow a reader to skip row groups that cannot satisfy a spatial query, which, paired with the spatial sorting from Milestone 2, is what makes partial reads of a GeoParquet file effective.

Completion is expected within 6 months of the award date.

## **Success**

### **Definition of done**

All four milestones are completed, package published on CRAN, and the joint blog post with the R Consortium is published.

## Measuring success

Correctness is measured against `{sf}`. Every operation shared with `{sf}` is tested to return the same result, and those tests run on every commit.

Performance is measured against the other engines available in R. The package's website includes a replication of the [duckspatial benchmark](#) to compare `geoarrowrs`, `sedonadb`, and `duckspatial`. In addition, there will be a `geoarrowrs` implementation of Apache Sedona [SpatialBench](#) for a reference.

## Future work

This work, particularly the compatibility with `{sf}` will prove how to make the `{sf}` package truly agnostic to the geometry engine provided. The findings in this work will directly lead to potential completion and development of the `{sdf}` R package.